Duas maneiras de definir agentes

Nos módulos 1 a 3, você criou agentes usando **código Python** no `agent.py`:

```
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='math_tutor_agent',
    description='Ajuda estudantes a aprender álgebra',
    instruction='Você é um orientador de matemática paciente...'
)
```

Ele é ótimo para desenvolvimento. No entanto, o ADK também permite definir agentes usando **arquivos de configuração YAML**, o que pode ser mais fácil para:

- Edição por não programadores
- Controle de versões e compartilhamento
- Experimentos rápidos
- Separação entre configuração e código

Neste módulo, você vai conhecer o **Agent Config**, um método do ADK com base em YAML para criar agentes.

Observação sobre o suporte à linguagem

Importante: o ADK é compatível com várias linguagens de programação:

- **Python**: suporte total ao código Python e ao Agent Config com YAML
- **Java**: compatível com agentes baseados em código Python (consulte o Guia de início rápido do Java, em inglês)

O Agent Config (YAML) é somente Python. Trecho extraído da documentação do ADK (em tradução livre):

> "**Linguagem de programação**: o recurso Agent Config é compatível apenas com código Python para ferramentas e outras funcionalidades que exigem código de programação."

Este módulo aborda o recurso Agent Config com YAML, que está disponível no ADK com Python.

Dois métodos para definir agentes

Referência: Documentação do ADK – Agent Config (em inglês)

Método 1: código Python (módulos 1 a 3)

Criado com: `adk create my_agent`

Gera:

```
my_agent/
├── agent.py      # Código Python que define o agente
├── __init__.py
└── .env
```

agent.py:

```python
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='assistant_agent',
    description='Um assistente colaborativo',
    instruction='Você é um assistente colaborativo.'
)
```

Método 2: configuração com YAML (este módulo)

Criado com: `adk create --type=config my_agent`

Gera:

```
my_agent/
├── root_agent.yaml   # Arquivo de configuração YAML
└── .env
```

root_agent.yaml:

```yaml
name: assistant_agent
model: gemini-2.5-flash
description: Um assistente colaborativo
instruction: Você é um assistente colaborativo.
```

Trecho extraído da documentação do ADK (em tradução livre):

> "O recurso Agent Config do ADK permite criar um fluxo de trabalho do ADK sem escrever código. O Agent Config usa um arquivo de texto no formato YAML com uma breve descrição do agente."

Como criar um agente com base em YAML

Referência: Documentação do ADK – Agent Config: criar um agente (em inglês)

Agora vamos criar o primeiro agente com base em YAML. O processo é parecido com o que você aprendeu no módulo 1, mas tem uma diferença fundamental: em vez de gerar um código Python, vamos criar um arquivo de configuração YAML.

Etapa 1: criar o projeto de agente

Execute o seguinte comando para criar um agente baseado em configuração:

```
adk create --type=config my_config_agent
```

Observe a flag `--type=config`. É ela que instrui o ADK a criar um agente baseado em YAML em vez do agente padrão em código Python.

Trecho extraído da documentação do ADK (em tradução livre):

> "Esse comando gera a pasta `my_agent/`, com os arquivos `root_agent.yaml` e `.env`."

Uma nova pasta `my_config_agent/` é exibida com:

- `root_agent.yaml`: a configuração do agente (em vez do `agent.py`)
- `.env`: variáveis de ambiente para chaves de API (igual ao módulo 1)

Etapa 2: configurar a chave de API

Assim como no módulo 1, você precisa configurar o arquivo `.env` com a chave de API. O processo é idêntico ao da etapa 7 do módulo 1: abra `my_config_agent/.env` e adicione `GOOGLE_API_KEY`.

Para refrescar a memória, consulte o Módulo 1 – Etapa 7: configure a chave de API.

Etapa 3: editar a configuração com YAML

Este é o pulo do gato. Em vez de escrever o código Python, você edita um arquivo YAML.

Abra o `root_agent.yaml` no editor de texto. O arquivo padrão gerado tem esta aparência:

```
# yaml-language-server:
$schema=https://raw.githubusercontent.com/google/adk-python/refs/heads/main/src/google
/adk/agents/config_schemas/AgentConfig.json
name: assistant_agent
model: gemini-2.5-flash
description: Um agente auxiliar que responde a perguntas dos usuários.
instruction: Você é um agente que responde a várias perguntas dos usuários.
```

Essa é uma configuração básica de agente com quatro parâmetros principais. Observe como o YAML é claro e legível em comparação com o código Python. Não há importações nem definições de função, apenas pares de chave-valor simples.

Vamos personalizá-la para criar o mesmo agente orientador de matemática que você criou no módulo 2. Edite o arquivo `root_agent.yaml` para que tenha esta aparência:

```
# yaml-language-server:
$schema=https://raw.githubusercontent.com/google/adk-python/refs/heads/main/src/google
/adk/agents/config_schemas/AgentConfig.json
name: math_tutor_agent
model: gemini-2.5-flash
description: Ajuda estudantes a aprender álgebra com orientação pelas etapas de
solução de problemas.
instruction: |
  Você é um orientador de álgebra paciente e motivador.

  Seu método de ensino:
  1. Quando um estudante faz uma pergunta, primeiro entenda a dificuldade dele
  2. Divida o problema em etapas menores e viáveis
  3. Oriente-o a encontrar a resposta em vez de responder diretamente
  4. Incentive o trabalho de raciocínio e conquista
  5. Use linguagem simples e evite jargões

  Sempre mantenha um tom calmo e apoiador. O aprendizado leva tempo e cada
  pergunta é uma oportunidade de crescer.
```

Conceitos básicos sobre a sintaxe YAML:

A barra vertical `|` após `instruction:` indica ao YAML que tudo o que vem a seguir é um texto de várias linhas. Esse recurso é perfeito para escrever instruções detalhadas que precisam de várias linhas. Em geral, não é preciso usar aspas ou caracteres de escape especiais. Basta escrever as instruções de forma natural.

Cada parâmetro está diretamente relacionado ao que você aprendeu no módulo 2:

- `name`: o identificador exclusivo do agente
- `model`: o LLM que será usado (`gemini-2.5-flash`)
- `description`: o que o agente faz (para que outros agentes entendam)
- `instrução`: o comportamento do agente (a parte mais importante)

Etapa 4: executar o agente baseado em YAML

Esta é a melhor parte: **a execução de um agente baseado em YAML é idêntica à de um agente em Python**. Todos os comandos que você aprendeu no módulo 3 funcionam da mesma maneira.

No diretório do espaço de trabalho (a pasta pai que contém o `my_config_agent`), execute:

```
adk web my_config_agent
```

Esse comando funciona da mesma forma que nos módulos 1 e 2. O ADK carrega o arquivo `root_agent.yaml`, cria o agente com base nessa configuração e inicia a interface da web.

Trecho extraído da documentação do ADK (em tradução livre):

> "Você pode executar o agente definido pelo Agent Config... `adk web`: executar a

interface da web do agente."

Quando a interface da web for aberta, interaja com o agente orientador de matemática assim como fez no módulo 2. Faça a pergunta "Como resolvo 2x + 5 = 13?" e veja o agente orientar você na solução do problema.

Quando usar YAML ou Python

Agora que você já conhece os dois métodos, talvez esteja se perguntando: "Quando devo usar a configuração com YAML em vez do código Python?". Os dois criam agentes que funcionam de forma idêntica, mas são adequados em cenários diferentes.

Aspecto	Configuração com YAML	Código Python
Facilidade de edição	✅ Muito fácil (arquivo de texto)	⚠️ Requer conhecimento de programação
Ideal para	Agentes simples, protótipos rápidos	Agentes complexos, lógica personalizada
Controle de versões	✅ Diferenças claras	⚠️ Mudanças no código
Flexibilidade	⚠️ Limitado a opções de configuração	✅ Controle programático total
Não programadores	✅ Podem editar	❌ Precisam de habilidades de programação
Recursos avançados	⚠️ Em breve	✅ Acesso total

Escolha YAML quando:

Você quer simplicidade e acessibilidade. A configuração com YAML é perfeita quando você cria agentes simples e quer manter tudo bem fácil. Não programadores podem tranquilamente editar um arquivo YAML para ajustar o comportamento do agente sem lidar com códigos, o que é excelente para colaboração entre equipes em que nem todos têm experiência em programação.

Você precisa fazer experimentos rápidos. Quer testar instruções diferentes ou alternar entre modelos? Basta editar o arquivo YAML e reiniciar o agente. Você não precisa de importações e definições de classe do Python.

A configuração deve ser separada do código. O método com YAML mantém a configuração do agente separada da lógica personalizada que você pode adicionar no futuro. Isso facilita controlar as diversas configurações em ambientes de desenvolvimento, teste e produção.

Escolha Python quando:

Você cria sistemas multiagente complexos. Como você vai aprender no curso 7, as

arquiteturas sofisticadas multiagente com delegação, agentes de fluxo de trabalho e orquestração personalizada exigem total capacidade do código Python. O método com YAML está limitado às opções básicas de configuração.

Você precisa de ferramentas e callbacks personalizados. Nos cursos 4 e 5, você vai aprender a adicionar ferramentas personalizadas e implementar callbacks. Esses recursos avançados exigem código Python e não podem ser expressos na íntegra apenas em YAML.

Você quer um controle programático. Às vezes, é necessário gerar as configurações do agente de maneira dinâmica, modificar o comportamento com base nas condições do ambiente de execução ou se integrar mais de perto a outras bibliotecas Python. O código Python oferece essa flexibilidade.

Exemplo: comparação lado a lado

Para entender a equivalência entre YAML e Python, vamos analisar o mesmo agente simples definido com os dois métodos. Esse agente de saudação tem os mesmos quatro parâmetros que usamos neste curso.

Código Python (agent.py):

```python
from google.adk.agents.llm_agent import Agent

root_agent = Agent(
    model='gemini-2.5-flash',
    name='greeting_agent',
    description='Um agente de saudação amigável',
    instruction='Cumprimente os usuários de modo afetuoso e profissional.'
)
```

Configuração com YAML (root_agent.yaml):

```yaml
name: greeting_agent
model: gemini-2.5-flash
description: Um agente de saudação amigável
instruction: Cumprimente os usuários de modo afetuoso e profissional.
```

Resultado: ambos produzem o mesmo comportamento do agente. Quando você executa qualquer uma das versões com `adk web`, um agente de saudação idêntico é retornado. A versão YAML é mais clara e fácil de ler: sem importações, parênteses nem aspas (na maioria dos casos), apenas uma configuração direta.

Suas conquistas

Parabéns! Agora você sabe que há **duas maneiras de definir agentes no ADK**, e que ambas são igualmente válidas, dependendo das suas necessidades.

Você começou pelo código Python nos módulos 1 a 3, em que criou os agentes escrevendo arquivos `agent.py` com importações e instanciação de classe. Dessa forma, você adquiriu experiência prática com a estrutura fundamental dos agentes.

Agora você descobriu a configuração com YAML, uma maneira mais simples de definir agentes sem código. Você aprendeu que o comando `adk create --type=config` gera um arquivo `root_agent.yaml` em vez do código Python, o que torna a criação de agentes acessível a não programadores.

Mais importante, você sabe que as duas abordagens produzem agentes idênticos. Seja em Python ou YAML, o orientador de matemática apresenta o mesmo comportamento. Os comandos `adk web`, `adk run` e `adk api_server` funcionam igualmente nas duas abordagens.

Você também aprendeu quando escolher cada método: YAML para simplicidade e prototipagem rápida e Python para sistemas multiagente complexos e recursos avançados, que você ainda vai conhecer em cursos futuros.

Por fim, você sabe que o **Agent Config é somente Python**, embora o próprio ADK seja compatível com várias linguagens de programação, incluindo Java. Conforme o framework evolui, o suporte a YAML pode abranger outras linguagens.

Pontos principais

O ADK oferece flexibilidade para definir agentes. Você só precisa escrever código se quiser. A configuração com YAML oferece uma alternativa clara e acessível para agentes simples.

A **configuração com YAML é perfeita** quando você precisa de experimentos rápidos, quando membros da equipe sem especialidade técnica precisam ajustar o comportamento do agente ou se quiser uma separação clara entre configuração e código. Você edita e salva um arquivo YAML, e depois reinicia o agente sem programação.

O **código Python se torna essencial** à medida que os agentes ficam mais sofisticados. Quando você adiciona ferramentas personalizadas (curso 4), implementa callbacks (curso 5) ou cria sistemas multiagente (curso 7), precisa da capacidade avançada do Python. Você sempre pode começar com YAML e migrar para Python quando necessário.

Os dois são métodos de primeira categoria do ADK. Não existe opção "certa" ou "errada". Escolha aquela que melhor se adapte às suas necessidades atuais. Lembre-se de que a equipe do ADK desenvolve constantemente o recurso Agent Config e aceita feedback.

Trecho extraído da documentação do ADK (em tradução livre):

> "O recurso Agent Config é experimental e tem algumas limitações conhecidas. Queremos saber sua opinião!"